Software Testing Portfolio

S2184546

January 28, 2025

1 Outline of the Software Being Tested

repository link
The software is a Java-based application designed to manage a drone delivery system for a pizza service called PizzaDronz (ILP 2023-2024). It calculates optimal drone flight paths to deliver orders efficiently while adhering to constraints such as no-fly zones and a limit on the number of pizzas in one order. The software integrates data dynamically from a RESTful API, processes JSON input for orders, and generates detailed output files, including delivery logs and flight paths in JSON and GeoJSON formats.

2 Learning Outcomes

1. **Analyze requirements to determine appropriate testing strategies** [15%]

   (a) **Range of requirements, functional requirements, measurable quality attributes, qualitative requirements**
   The full list of requirements can be found in the requirements file. I have analysed my project and found a wide range of functional, measurable, and qualitative requirements. For example. Functional: "Drones must avoid no-fly zones", Measurable: "Must output flight plan in less than 60s", Qualitative: "Java 18 compliance". There are also some security and robustness requirements, but these could be considered sub-categories of functional.

   (b) **Level of requirements, system, integration, unit**
   In the same file, I have indicated what level of tests are required. For example, "Drones must avoid no-fly zones" would mainly require system level tests, since it is important that this high priority requirement is verified in the final system. Unit tests may also be used to just test the path finding, since order validation would not have a significant impact on this requirement (though it could still affect it, for example, if a restaurant is inside a no-fly zone).

   (c) **Identifying test approach for chosen attributes**
   Test approaches are indicated in the requirements file. For example, the test approach for "Must be able to dynamically fetch data from a REST API" would be integration tests that involve using a mock API that mimics the specification of the real API, and testing the systems ability to request, receive, and process the data

   (d) **Assess the appropriateness of your chosen testing approach**
   There are numerous constraints that impact the effectiveness of my testing approaches. For the requirement "Must output flight plan in less than 60s", due to constraints on the hardware that is available to me, I can only test the requirement on my own limited number of computers, as well as some cloud servers (such as the servers used for GitHub Actions). This means that it cannot be guaranteed that the flight plan will be outputted on all computers in under 60s. Furthermore, the system specifications do not indicate the maximum number of orders, restaurants, or no-fly zones, all of which would affect the processing time. For the requirement "Code Quality", I can test that the code follows a set standard (such as using JavaDocs), but this does not guarantee the code's readability and maintainability. This would require the subjective opinion of other programmers, which I do not have access to.

2. **Design and implement comprehensive test plans with instrumented code** [15%]

   (a) **Construction of the test plan**
   The test plan is available in the file

   (b) **Evaluation of the quality of the test plan**
   A notable area from my requirements that are missing in my test plan is security. The API that the system communicates with is unauthenticated, meaning that any security related to the transmission of data (such as using HTTPS) would be meaningless, since anyone can access the data. I have also not included input sanitization in my test plan, since the users of the software would be inside the pizza delivery company (the software would not be used by the public), there is less of a risk of malformed inputs or injection attacks (which makes this requirement a lower priority).

   (c) **Instrumentation of the code**
   Scaffolding and instrumentation for some of the requirements is in the test plan. For example, I used the WireMock tool to build mock APIs to test how well my code integrate with the API.

   (d) **Evaluation of the instrumentation**
   In my test plan, I originally intended to implement tests that would generate random configurations of no-fly zones. Due to time constraints, I opted to just use hand-crafted test cases, where I tried to make a mix of difficulties. I did successfully implement logs using log4j2 framework throughout many parts of the program, which provide insight into what parts of the system successfully run and which parts have errors during debugging. I Also performed manual system tests on the packaged jar file, to ensure that the software was packaged correctly.

3. **Apply a wide variety of testing techniques and compute test coverage and yield according to a variety of criteria** [default 20%]

   (a) **Range of techniques**
   I used functional testing to validate that the flight planner avoids no-fly zones and fetches valid REST API data. I used structural testing to find branches that were not covered by testing, in particular the order validation class where there are many branches for different invalid orders. I used Boundary Value and partition testing, in particular to test that the number of pizzas per order is between 0 and 4 and to test the geometry methods (specifically the isInRegion method). I used performance testing to test the speed of the system under a varity of different configurations of orders and no-fly zones.

   (b) **Evaluation criteria for the adequacy of the testing** The performance tests were largely pessimistic, with no-fly zones constructed to cause maximum disruption to the algorithm in order to ensure the computation times was under 60s. The integration tests with the REST API were simplified and only used a mock API due to not having access to the real API. They also did not consider high stress scenarios, network failures, and malformed API responses. The order validation tests were optimistic, since they assumed that there was at most one error in each order (this was given in the project specification).

   (c) **Results of testing**
   Results shown in images and screenshots (the png files in the repository). All tests passes. Some issues that were uncovered and solved were: Paths going though no-fly zones, the credit card date checker causing a crash on certain days, and execution time being over 60s.

   (d) **Evaluation of the results**
   Looking at the performance chart, we can see that the computation time increases as the number and complexity of the no-fly zones increase. On the other hand, performance is not significantly impacted by the number of orders. This is because the path-finding algorithm uses caches for the paths, meaning that once the limited number of possible paths are computed, any additional orders will not require additional path-finding computation. In fact, using the built-in intellij performance profiler, I found that the pathfinding represents about 99.0% of all the computation, with the output of the files taking the second most and the order validation taking only 10ms (compared to 9820ms for the path-finding). This explains why changing the number of orders does not significantly change performance.

4. **Evaluate the limitations of a given testing process, using statistical methods where appropriate, and summarise outcomes** [25%]

   (a) **Identifying gaps and omissions in the testing process**
   One of the main gaps in my testing is the lack of API performance testing. This would involve checking the latency and throughput of each response (throughput may be different, since the API currently sends all the orders for a day in a single response) and analysing how this affects overall performance. This could potentially meaningfully affect the results of my performance testing, since the current performance tests are performed using a mock API (which has a latency of near zero). Another aspect of this would be stress testing, which would look at how many requests and responses can be handled in a specific time period. The reason that this testing is absent is because I do not have access to the real API (I am using my project from last year) and creating my own API that mimics the previous one would take far too much time. Another aspect that is missing is more data points for my performance testing. Currently, I only tested 6 different cases with varying number of orders and no-fly zones. This means that my performance data may not be that accurate, due to the limited data. The reason for this gap in my testing is that I do not have scaffolding that can automatically generate test cases (though this would be very difficult to do for no-fly zone difficulty, since it depends on more than quantity). This means that I have to hand craft the test cases, which would take too much time if I were create more than what I currently have.

   (b) **Identifying target coverage/performance levels for the different testing procedures**
   My target code coverage was 80% line and branch coverage. My target mutation kill rate was 70%. My target performance was under 60s.

   (c) **Discussing how the testing carried out compares with the target levels**
   My code coverage easy met my targets, getting 92% line coverage and 91% branch coverage. However, my mutation target was barely met, having a kill rate of 72%. This suggests that my tests have a wide breadth, but do not deeply test the code. My performance target was also met, with my hardest test case taking just over 2 seconds to complete. I also validated my performance by creating an incalculable test case (where no path can be found), and found that the timeout mechanism in my code works correctly, capping the time spent on this test to under 12 seconds.

   (d) **Discussion of what would be necessary to achieve the target level**
   I did successfully meet all my targets, but the aspect that was the weakest was my mutation testing results. This suggests that I should improve the quality of my tests by adding more assertions or making existing assertions more strict.

5. **Conduct reviews, inspections, and design and implement automated testing processes** [25%]

   (a) **Identify and apply review criteria to selected parts of the code and identify issues in the code**
   When I was reviewing the performance of the pathfinding, I found that the software was repentantly calculating the same paths. I noticed this when using the geojson.io visualisation tool to see the paths. To fix this, I implemented path caching so that future paths that have the same start and end point use the previously calculated path. I also used test coverage measurements to inform me of parts of the code that is not being tested, which resulted in extra tests being written which helped to find bugs. I also used Intellij's built-in code inspection for identifying things such as code duplication, unused imports, and unhandled exceptions. Some limitations were a lack of stress testing and a lack of peer code reviews.

   (b) **Construct an appropriate CI pipeline for the software**
   A CI pipeline was implemented in GitHub using Maven to get dependencies, build the application, test the application, and package the application into a jar. Pitest mutation testing was also put into the CI pipeline.

   (c) **Automate some aspects of the testing**
   All the units tests are run automatically when pushing a commit, preventing the integration

of the changes if any tests fail. Due to time constraints, I was not able to add Jacoco code coverage tool to the CI pipeline, but this would be one improvement that could be made.

(d) **Demonstrate the CI pipeline functions as expected**
If a commit causes any of the tests to fail, then it is rejected. This prevents changes to the code that would break or introduce bugs to the software. This would be particularly useful for large development teams. The use of mutation testing in the pipeline gives feedback on how adequate the testing is for new code, or how changes to tests affect the quality of the testing. This could be optimised to perform mutation testing less often, since mutation testing is likely not necessary for every commit. As mentioned before, I was not able to add Jacoco to the pipeline, but this would provide similar benefits to the mutation testing, but would take much less time (making it more viable to use in every commit).